

Федеральное государственное автономное образовательное учреждение
высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Автоматизированные системы обработки информации и управления»

Направление подготовки: 09.03.02 Информационные системы и технологии

ОТЧЕТ ПО ПРОЕКТНОЙ ПРАКТИКЕ

Студент: Ципцин Егор Алексеевич, группа: 251-331

Место прохождения практики: Московский Политех, проектная деятельность
«NeoPixel. Современное аддитивное оборудование»

Куратор проектной деятельности: Богуш Максим Михайлович

Отчет принят с оценкой _____ Дата _____

Руководитель практики: _____

Москва 2026

ОГЛАВЛЕНИЕ

Введение

1. Общая информация о проектной практике
 - 1.1. Название и тематика проекта
 - 1.2. Цели и задачи практики
 - 1.3. Общая характеристика проектной среды
2. Индивидуальная задача в проекте NeoPixel
 - 2.1. Постановка задачи «Реализация базового управления Z-осью»
 - 2.2. Аппаратно-программная основа задачи
 - 2.3. Программная логика управления Z-осью
 - 2.4. Калибровка, homing и координатная модель
 - 2.5. Значение результата для дальнейшего развития принтера
3. Разработка сайта проектной деятельности
 - 3.1. Структура и содержание сайта
 - 3.2. Графические материалы и медиа-сопровождение
 - 3.3. Переход к Vite и запуск через ngrok
4. Подготовка документации
5. Вариативная часть практики: Python Simple Web Server
 - 5.1. Постановка вариативного задания
 - 5.2. Архитектура учебного HTTP-сервера
 - 5.3. Проверка и демонстрация результата
 - 5.4. Отражение вариативной части на сайте
6. Описание достигнутых результатов

Заключение

Список использованных источников

Приложения

ВВЕДЕНИЕ

Проектная практика была выполнена в рамках образовательной программы Московского Политехнического университета и была связана с двумя взаимодополняющими направлениями работы: участием в проектной деятельности по теме NeoPixel и выполнением индивидуальной вариативной части, направленной на практическое изучение сетевого программирования и принципов работы HTTP-сервера.

Основная проектная часть была связана с учебным проектом «NeoPixel. Современное аддитивное оборудование». Проект относится к области малогабаритной фотополимерной 3D-печати и предполагает разработку, анализ и доработку программно-аппаратных компонентов малого фотополимерного принтера. В рамках данной работы особое внимание было уделено подсистеме вертикального позиционирования, то есть базовому управлению Z-осью.

Помимо анализа и описания проектной задачи, был разработан статический сайт, предназначенный для представления проекта, участников, журнала прогресса, ресурсов и вариативной части практики. Первоначально сайт был оформлен как набор статических HTML/CSS/JS-файлов, после чего для удобства запуска и демонстрации был добавлен Vite.

Вариативная часть практики была посвящена реализации учебного HTTP-сервера на Python по теме «Python: A Simple Web Server» из подборки build-your-own-х. Реализация была выполнена без использования готовых веб-фреймворков, на стандартной библиотеке Python и модуле socket.

1. ОБЩАЯ ИНФОРМАЦИЯ О ПРОЕКТНОЙ ПРАКТИКЕ

1.1. Название и тематика проекта

Проектная деятельность выполнялась в рамках проекта «NeoPixel. Современное аддитивное оборудование». Тематика проекта связана с разработкой и доработкой малого фотополимерного 3D-принтера. В отличие от абстрактных учебных задач, данный проект находится на стыке программирования микроконтроллеров, силовой электроники, механики, пользовательского интерфейса и технологических процессов аддитивного производства.

Фотополимерный 3D-принтер представляет собой устройство, в котором модель формируется послойно за счет локальной засветки жидкого фотополимера ультрафиолетовым излучением. Для корректной работы такого устройства важны не только UV-источник и дисплейный модуль, но и стабильная механика перемещения платформы.

1.2. Цели и задачи практики

Цель практики заключалась в комплексном оформлении результатов проектной деятельности и индивидуальной работы в виде проверяемого репозитория, сайта, документации и программного результата вариативной части.

1. изучить исходный репозиторий проекта NeoPixel / Mini_Pixel_proj;
2. определить текущую стадию готовности проекта и выделить реализованные крупные задачи;
3. выбрать индивидуальную задачу в рамках существующей логики проекта;
4. подробно описать задачу «Реализация базового управления Z-осью»;
5. создать статический сайт по проектной деятельности;
6. добавить страницы о проекте, участниках, журнале, ресурсах и вариативном задании;
7. подготовить графические материалы, схемы и медиа-блоки;
8. добавить Vite для удобного запуска сайта и сборки;
9. описать запуск сайта, включая вариант публикации через ngrok;
10. подготовить документацию в директории docs;
11. изучить и реализовать вариативную часть «Python: A Simple Web Server»;
12. сформировать итоговый отчет по всей проделанной работе.

1.3. Общая характеристика проектной среды

Практика выполнялась в образовательной среде Московского Политехнического университета. Проектная деятельность в таком формате ориентирована не только на получение итогового программного артефакта, но и на развитие инженерного мышления: умения анализировать уже существующий код, выявлять степень готовности проекта, описывать ограничения, формулировать дальнейшие шаги и представлять результат в понятной для внешнего наблюдателя форме.

Важной особенностью практики стало сочетание embedded-направления и web-направления. С одной стороны, проект NeoPixel связан с микроконтроллерной логикой ESP32-S3, драйвером A4988, шаговым двигателем NEMA17 и концевым выключателем. С другой стороны, результат практики должен быть оформлен как сайт, документация и вариативное сетевое приложение.

2. ИНДИВИДУАЛЬНАЯ ЗАДАЧА В ПРОЕКТЕ NEOPIXEL

2.1. Постановка задачи «Реализация базового управления Z-осью»

В качестве индивидуальной задачи была выбрана тема «Реализация базового управления Z-осью». Эта задача является логически обоснованной, поскольку именно управление вертикальной платформой уже присутствовало в репозитории в виде наиболее развитой программной заготовки. В рамках работы необходимо было понять инженерный смысл функций: как микроконтроллер управляет драйвером, как формируются импульсы, как задается направление, как используется концевой выключатель и почему homing является обязательной операцией.

Задача была рассмотрена как аппаратно-программный интерфейс между технологической логикой фотополимерной печати и физическим перемещением платформы. На уровне печатного процесса требуется переместить платформу на определенное расстояние. На уровне прошивки это превращается в набор STEP-импульсов, установку линии DIR, включение драйвера EN и контроль концевого выключателя.

2.2. Аппаратно-программная основа задачи

Элемент

Назначение

ESP32-S3

Центральный микроконтроллер, формирующий управляющие сигналы.

A4988

Драйвер шагового двигателя, принимающий STEP, DIR и EN.

NEMA17

Шаговый двигатель, обеспечивающий перемещение платформы по оси Z.

Концевой выключатель

Датчик нулевой позиции, используемый для калибровки.

STEP

Сигнал отдельного шага двигателя.

DIR

Сигнал направления движения.

EN

Сигнал включения или отключения драйвера.

В рамках задачи рассматривалась типовая связка ESP32-S3, A4988, NEMA17 и концевой выключатель. Такая архитектура характерна для малых станков и 3D-принтеров, где контроллер не измеряет позицию напрямую, а ведет учет выполненных шагов.

2.3. Программная логика управления Z-осью

В репозитории проекта логика управления оформлена через класс StepMotor. В нем предусмотрены методы begin, enable, disable, move, moveMM, homing, printLayerSequence, isEndstopTriggered, getPositionSteps, getPositionMM и isHomed. Такой набор методов показывает переход от низкоуровневой работы с GPIO к более осмысленным операциям движения.

Метод `begin` отвечает за настройку GPIO: выходы STEP, DIR и EN переводятся в режим `output`, а вход концевика настраивается с подтяжкой. Метод `enable` активирует драйвер, а `disable` отключает его. Метод `move` генерирует заданное количество импульсов STEP в выбранном направлении, одновременно проверяя концевик при движении вниз. Метод `moveMM` добавляет инженерную абстракцию: пользователь или дальнейшая логика может задавать расстояние в миллиметрах, а прошивка переводит его в шаги через параметр `STEPS_PER_MM`.

Метод `printLayerSequence` является заготовкой под будущую последовательность печати: платформа поднимается на величину отрыва и высоты слоя, затем возвращается вниз на величину отрыва. Пока эта логика не связана с реальной UV-экспозицией, но она важна архитектурно.

2.4. Калибровка, homing и координатная модель

Особое значение имеет процедура `homing`. Шаговый двигатель сам по себе не сообщает контроллеру абсолютное положение платформы. После включения устройства микроконтроллер не знает, где физически находится ось Z. Поэтому необходимо выполнить поиск опорной точки, для чего используется концевой выключатель.

В изученной реализации `homing` включает несколько этапов: движение вниз до срабатывания концевика, откат вверх, повторное медленное приближение к концевiku и финальный откат. После завершения процедуры текущая позиция принимается за $Z = 0$, а флаг `_isHomed` переводится в состояние `true`.

С инженерной точки зрения `homing` является механизмом синхронизации программной модели с физической механикой. Он снижает риск выхода платформы за допустимые пределы, позволяет восстановить координатную систему после включения и создает основу для повторяемого печатного процесса.

2.5. Значение результата для дальнейшего развития принтера

Реализация базового управления Z-осью не делает устройство полноценным фотополимерным принтером, однако создает один из необходимых нижних уровней. Без предсказуемого движения платформы невозможно проверить

корректность UV-экспозиции, вывод изображений слоев, пользовательские настройки и сценарии печати.

3. РАЗРАБОТКА САЙТА ПРОЕКТНОЙ ДЕЯТЕЛЬНОСТИ

3.1. Структура и содержание сайта

Для представления результатов проектной деятельности был создан статический сайт в директории site. Сайт был построен как многостраничный ресурс, отражающий общую информацию о NeoPixel, индивидуальную задачу, участников, журнал прогресса, полезные ресурсы и вариативное задание.

Страница

Назначение

index.html

Домашняя страница с аннотацией NeoPixel и визуальным представлением проекта.

about.html

Описание проекта, аппаратной рамки и текущей логики Z-оси.

participants.html

Список участников группы.

journal.html

Журнал прогресса с последовательными этапами работы.

resources.html

Ссылки на Московский Политех, проектную деятельность и техническую документацию.

variant.html

Страница вариативного задания, посвященная учебному HTTP-серверу.

3.2. Графические материалы и медиа-сопровождение

В сайт были добавлены локальные SVG-материалы: визуальный hero-блок, схема управления Z-осью, функциональная карта NeoPixel, диаграмма

послойной последовательности печати, анимированная модель движения платформы и схема работы учебного web server. Такие материалы помогают объяснить техническую структуру проекта.

3.3. Переход к Vite и запуск через ngrok

Для повышения удобства разработки был добавлен Vite. Это позволило запускать сайт командой `npm run dev`, использовать локальный dev-сервер, выполнять сборку `npm run build` и получать директорию `dist` для публикации. Также был рассмотрен сценарий временной публикации сайта через ngrok.

4. ПОДГОТОВКА ДОКУМЕНТАЦИИ

В директории `docs` была подготовлена документация, отражающая как проектную, так и техническую часть практики. Центральным файлом стал `docs/README.md`, где описано назначение каждого документа. Отдельный файл `docs/site.md` содержит руководство по запуску сайта: от клонирования репозитория до открытия страницы в браузере, включая `npm install`, `npm run dev`, `npm run build`, `npm run preview` и публикацию через ngrok.

Документ `docs/неорixel_task.md` подробно описывает индивидуальную задачу «Реализация базового управления Z-осью». Также в документации присутствуют материалы вариативной части: `docs/variant_task.md` и `docs/variant_journal.md`.

5. ВАРИАТИВНАЯ ЧАСТЬ ПРАКТИКИ: PYTHON SIMPLE WEB SERVER

5.1. Постановка вариативного задания

В качестве вариативной части была выбрана тема «Python: A Simple Web Server» из подборки `codecrafters-io/build-your-own-x`. Целью было не использовать готовый фреймворк, а практически воспроизвести базовую механику web server: принять TCP-подключение, прочитать HTTP-запрос, разобрать его, выбрать обработчик, сформировать ответ и отправить его клиенту.

5.2. Архитектура учебного HTTP-сервера

Сервер реализован в файле `src/simple_web_server.py`. Основной класс `SimpleWebServer` отвечает за открытие TCP-сокета, прием соединений, запуск

обработчиков клиентов, чтение запроса, маршрутизацию и отправку ответа. Для представления данных используются структуры `HttpRequest` и `HttpResponse`.

Файл или директория

Назначение

`src/simple_web_server.py`

Основной код учебного HTTP-сервера.

`src/public/index.html`

Главная демонстрационная страница сервера.

`src/public/about.html`

Страница с описанием возможностей сервера.

`src/public/styles.css`

Стили для демонстрационных страниц и страниц ошибок.

`src/public/notes/readme.txt`

Тестовый файл для проверки листинга директории.

Функционально сервер поддерживает методы GET и HEAD, отдает статические файлы, автоматически возвращает `index.html` для директорий, генерирует листинг директории при отсутствии `index.html`, возвращает HTML-страницы ошибок, защищается от path traversal и содержит JSON-маршруты `/api/time` и `/api/echo`.

5.3. Проверка и демонстрация результата

Для проверки реализации была предусмотрена компиляция Python-файла и набор HTTP-сценариев. Сервер запускался командой:

```
python3 src/simple_web_server.py --host 127.0.0.1 --port 8080 --root src/public
```

Запрос

Ожидаемый результат

Смысл проверки

/

200 OK

Отдается index.html.

/about.html

200 OK

Отдается статическая HTML-страница.

/notes/

200 OK

Генерируется листинг директории.

/api/time

200 OK, JSON

Проверяется динамический JSON-ответ.

/missing.html

404 Not Found

Проверяется обработка отсутствующего ресурса.

/../../README.md

403 Forbidden

Проверяется защита от path traversal.

POST /

405 Method Not Allowed

Проверяется отклонение неподдерживаемого метода.

5.4. Отражение вариативной части на сайте

После подготовки вариативного задания на основной сайт была добавлена отдельная страница «Вариативное задание». На ней описаны постановка задачи, сетевой уровень, HTTP-логика, отдача статических файлов, JSON API,

структура реализации и команда запуска. Также была добавлена схема работы web server.

6. ОПИСАНИЕ ДОСТИГНУТЫХ РЕЗУЛЬТАТОВ

1. изучен репозиторий проекта NeoPixel / Mini_Pixel_proj;
2. выделена индивидуальная задача «Реализация базового управления Z-осью»;
3. подготовлено академическое описание аппаратной и программной основы Z-оси;
4. создан многостраничный сайт проекта в директории site;
5. добавлены страницы: домашняя, «О проекте», «Участники», «Журнал», «Ресурсы», «Вариативное задание»;
6. заполнен список участников проекта и отмечен куратор;
7. подготовлены локальные SVG-схемы и медиа-анимация;
8. добавлен Vite, package.json, package-lock.json, vite.config.js и сценарии запуска;
9. настроена поддержка ngrok-доменов через server.allowedHosts;
10. подготовлена документация docs/README.md, docs/site.md и docs/neopixel_task.md;
11. разработан учебный HTTP-сервер на Python в директории src;
12. подготовлены docs/variant_task.md и docs/variant_journal.md;
13. вариативная часть отражена на отдельной странице сайта;
14. подготовлен итоговый отчет в формате DOCX.

ЗАКЛЮЧЕНИЕ

В результате проектной практики была выполнена работа, объединяющая анализ embedded-проекта, создание сайта, подготовку документации и реализацию вариативного сетевого приложения. Основная проектная часть была связана с NeoPixel и задачей базового управления Z-осью малого фотополимерного 3D-принтера. Эта задача была рассмотрена как важнейшая исполнительная подсистема, без которой невозможна дальнейшая реализация полноценного печатного процесса.

Разработка сайта позволила оформить проект в наглядном виде. Были добавлены разделы о проекте, участниках, журнале, ресурсах и вариативной части. Использование Vite сделало запуск сайта более удобным, а сценарий с ngrok позволил временно раздавать сайт во внешний интернет.

Вариативная часть практики расширила технический диапазон работы. Реализация простого HTTP-сервера на Python позволила изучить базовые механизмы сетевого взаимодействия без абстракций фреймворков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Репозиторий практики: <https://github.com/dehorik/practice-2026>
2. Репозиторий проекта NeoPixel / Mini_Pixel_proj:
https://github.com/MEISSSTER/Mini_Pixel_proj
3. Репозиторий build-your-own-x: <https://github.com/codecrafters-io/build-your-own-x>
4. A Simple Web Server, 500 Lines or Less: <https://aosabook.org/en/500L/a-simple-web-server.html>
5. Документация ESP-IDF для ESP32-S3:
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/>
6. Документация PlatformIO: <https://docs.platformio.org/>
7. Документация LVGL: <https://docs.lvgl.io/>
8. Pololu A4988 Stepper Motor Driver Carrier:
<https://www.pololu.com/product/1182>
9. NEMA 17 Stepper Motor: https://reprap.org/wiki/NEMA_17_Stepper_motor
10. Python socket documentation: <https://docs.python.org/3/library/socket.html>

11.MDN Web Docs. HTTP overview:

<https://developer.mozilla.org/docs/Web/HTTP/Guides/Overview>

12.Vite documentation: <https://vite.dev/>

13.ngrok documentation: <https://ngrok.com/docs/>

14.Git documentation: <https://git-scm.com/doc>

15.GitHub Docs: <https://docs.github.com/>

ПРИЛОЖЕНИЯ

Приложение А. Структура репозитория

practice-2026/

README.md

docs/

README.md

site.md

neopixel_task.md

variant_task.md

variant_journal.md

site/

index.html

about.html

participants.html

journal.html

resources.html

variant.html

assets/

package.json

vite.config.js

src/

```
simple_web_server.py
```

```
public/
```

```
reports/
```

Приложение Б. Основные команды запуска сайта

```
git clone https://github.com/dehorik/practice-2026
```

```
cd practice-2026/site
```

```
npm install
```

```
npm run dev
```

```
npm run build
```

```
ngrok http 5173
```

Приложение В. Команда запуска вариативной части

```
python3 src/simple_web_server.py --host 127.0.0.1 --port 8080 --root src/public
```

Приложение Г. Краткая характеристика индивидуальной задачи

Индивидуальная задача «Реализация базового управления Z-осью» была связана с анализом и описанием программно-аппаратной логики управления вертикальным перемещением платформы фотополимерного 3D-принтера NeoPixel. В рамках задачи были рассмотрены микроконтроллер ESP32-S3, драйвер A4988, шаговый двигатель NEMA17, концевой выключатель, STEP/DIR/EN-сигналы, homing, координатная модель и переход от шагов к миллиметрам.

Приложение Д. Ссылка на Github-репозиторий

<https://github.com/dehorik/practice-2026>